

LCMFreq

Khai thác tập phổ biến bằng Backtracking, OccurrenceDeliver và Hypercube

Nhóm 01, CSC14004

HCMUS, học kỳ 2, năm học 2025–2026

Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Đặt vấn đề

Cho tập item $\mathcal{I}$ và database giao dịch

$$\mathcal{D} = \{T_1, \dots, T_m\}, T_i \subseteq \mathcal{I}.$$

Denotation: $\mathcal{T}(P) = \{T_i \in \mathcal{D} : P \subseteq T_i\}.$

Frequency / support: $\text{sup}(P) = |\mathcal{T}(P)|.$

P là **frequent** nếu $\text{sup}(P) \geq \sigma.$

Hai tính chất nền tảng

$$\mathcal{T}(P \cup Q) = \mathcal{T}(P) \cap \mathcal{T}(Q)$$

$$P \subseteq Q \Rightarrow \mathcal{T}(Q) \subseteq \mathcal{T}(P)$$

TID	Items
T_1	A, B, C, D
T_2	A, B, D
T_3	A, B, C
T_4	B, C
T_5	A, B, C, D

$$\sigma = 2: \text{sup}(\{A, C\}) = 3 \geq 2 \Rightarrow \text{frequent}.$$

Đơn điệu: P không frequent $\Rightarrow$ mọi superset của P cũng không frequent.

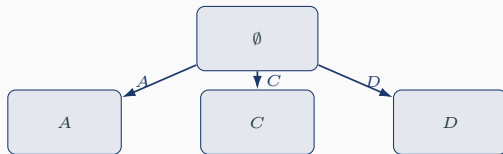
tail(P) = item chỉ số lớn nhất trong P .
Backtracking chỉ thêm $e > \text{tail}(P)$.

 $\emptyset$

Đơn điệu: P không frequent $\Rightarrow$ mọi superset của P cũng không frequent.

tail(P) = item chỉ số lớn nhất trong P .
Backtracking chỉ thêm $e > \text{tail}(P)$.

Hệ quả: mỗi itemset có đúng một đường sinh duy nhất từ $\emptyset$ — không sinh trùng.

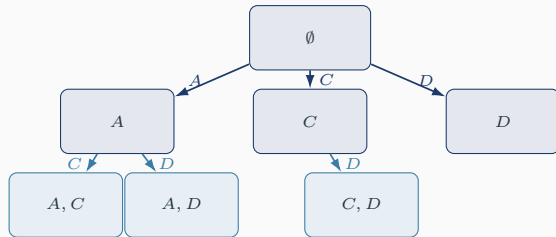


Tính đơn điệu $\Rightarrow$ cây sinh itemset không trùng

Đơn điệu: P không frequent $\Rightarrow$ mọi superset của P cũng không frequent.

tail(P) = item chỉ số lớn nhất trong P .
Backtracking chỉ thêm $e > \text{tail}(P)$.

Hệ quả: mỗi itemset có đúng một đường sinh duy nhất từ $\emptyset$ — không sinh trùng.

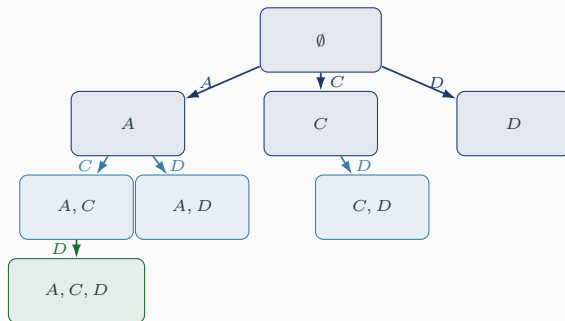


Tính đơn điệu $\Rightarrow$ cây sinh itemset không trùng

Đơn điệu: P không frequent $\Rightarrow$ mọi superset của P cũng không frequent.

tail(P) = item chỉ số lớn nhất trong P .
Backtracking chỉ thêm $e > \text{tail}(P)$.

Hệ quả: mỗi itemset có đúng một đường sinh duy nhất từ $\emptyset$ — không sinh trùng.



$\{A, C, D\}$ chỉ sinh được qua $A \rightarrow C \rightarrow D$, không qua $C \rightarrow A \rightarrow D$ hay $D \rightarrow A \rightarrow C$.

Khung Backtracking

Apriori vs. Backtracking

Apriori (level-by-level)

sinh $\mathcal{D}_k$ từ toàn bộ $\mathcal{D}_{k-1}$

phải giữ cả $\mathcal{D}_{k-1}$ trong bộ nhớ

+ dễ đếm support theo lô

Backtracking (DFS)

sinh đệ quy từ nghiệm hiện tại P

chỉ giữ P trên ngăn xếp

— tiết kiệm bộ nhớ

– đếm support từng candidate riêng lẻ, tốn thời gian

“Apriori algorithms use much memory for storing $\mathcal{D}_k \dots$ backtracking algorithms use less memory $\dots$ However, apriori algorithms have advantages for the frequency counting.” — Uno, Kiyomi, Arimura (2004)

LCMFreq = khung **Backtracking** + hai kỹ thuật bù lại điểm yếu đếm support.

```
BackTracking(P):  
    output P  
    for e in I,  $e > \text{tail}(P)$ :  
        if P union {e} la frequent:  
            BackTracking(P union {e})
```

Mỗi lượt gọi: xuất P , rồi thử mở rộng với từng $e > \text{tail}(P)$.

```
BackTracking(P):  
    output P  
    for e in I, e > tail(P):  
        if P union {e} la frequent:  
            BackTracking(P union {e})
```

Mỗi lượt gọi: xuất P , rồi thử mở rộng với từng $e > \text{tail}(P)$.

LCMFreq giữ nguyên khung này, chỉ thay hai bước bên trong:

- **OccurrenceDeliver** — đếm support nhanh hơn

```
BackTracking(P):  
    output P  
    for e in I, e > tail(P):  
        if P union {e} la frequent:  
            BackTracking(P union {e})
```

Mỗi lượt gọi: xuất P , rồi thử mở rộng với từng $e > \text{tail}(P)$.

LCMFreq giữ nguyên khung này, chỉ thay hai bước bên trong:

- [OccurrenceDeliver](#) — đếm support nhanh hơn
- [HypercubeDecomposition](#) — xuất theo lô



P

- **1. Backtracking:** prefix P , chỉ thêm $e > \text{tail}(P)$.



- **1. Backtracking:** prefix P , chỉ thêm $e > \text{tail}(P)$.
- **2. OccurrenceDeliver:** duyệt $\mathcal{T}(P)$ một lần, rải vào bucket của mọi candidate cùng lúc.



- **1. Backtracking:** prefix P , chỉ thêm $e > \text{tail}(P)$.
- **2. OccurrenceDeliver:** duyệt $\mathcal{T}(P)$ một lần, rải vào bucket của mọi candidate cùng lúc.
- **3. Hypercube:** item không làm giảm denotation $\Rightarrow$ xuất cả tổ hợp một lượt, không đệ quy riêng.



- **1. Backtracking:** prefix P , chỉ thêm $e > \text{tail}(P)$.
- **2. OccurrenceDeliver:** duyệt $\mathcal{T}(P)$ một lần, rải vào bucket của mọi candidate cùng lúc.
- **3. Hypercube:** item không làm giảm denotation $\Rightarrow$ xuất cả tổ hợp một lượt, không đệ quy riêng.

Đếm support hiệu quả

$\mathcal{T}(P_1 \cup P_2) = \mathcal{T}(P_1) \cap \mathcal{T}(P_2)$ tính trong $O(|\mathcal{T}(P_1)| + |\mathcal{T}(P_2)|)$

$\mathcal{T}(P_1)$



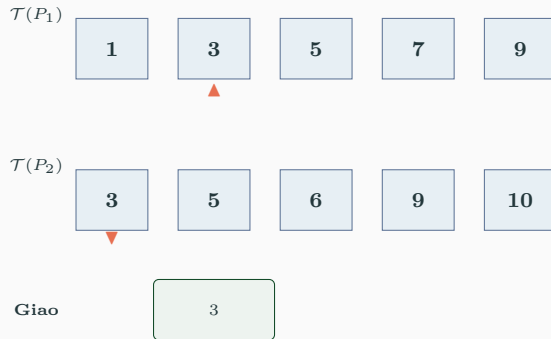
$\mathcal{T}(P_2)$



Giao

$i=1, j=1: 1 < 3 \Rightarrow$ tăng i .

$\mathcal{T}(P_1 \cup P_2) = \mathcal{T}(P_1) \cap \mathcal{T}(P_2)$ tính trong $O(|\mathcal{T}(P_1)| + |\mathcal{T}(P_2)|)$



$i=2, j=1: 3 = 3 \Rightarrow$ **xuất 3**, tăng cả hai.

$\mathcal{T}(P_1 \cup P_2) = \mathcal{T}(P_1) \cap \mathcal{T}(P_2)$ tính trong $O(|\mathcal{T}(P_1)| + |\mathcal{T}(P_2)|)$

$\mathcal{T}(P_1)$



$\mathcal{T}(P_2)$



Giao



$i=3, j=2: 5 = 5 \Rightarrow$ **xuất 5**, tăng cả hai.

$\mathcal{T}(P_1 \cup P_2) = \mathcal{T}(P_1) \cap \mathcal{T}(P_2)$ tính trong $O(|\mathcal{T}(P_1)| + |\mathcal{T}(P_2)|)$

$\mathcal{T}(P_1)$



$\mathcal{T}(P_2)$



Giao



$i=4, j=3: 7 > 6 \Rightarrow$ tăng j .

$\mathcal{T}(P_1 \cup P_2) = \mathcal{T}(P_1) \cap \mathcal{T}(P_2)$ tính trong $O(|\mathcal{T}(P_1)| + |\mathcal{T}(P_2)|)$

$\mathcal{T}(P_1)$



$\mathcal{T}(P_2)$



Giao



$i=4, j=4: 7 < 9 \Rightarrow$ tăng i .

Down project: giao hai danh sách TID bằng hai con trỏ

$$\mathcal{T}(P_1 \cup P_2) = \mathcal{T}(P_1) \cap \mathcal{T}(P_2) \quad \text{tính trong } O(|\mathcal{T}(P_1)| + |\mathcal{T}(P_2)|)$$

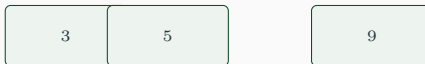
$\mathcal{T}(P_1)$



$\mathcal{T}(P_2)$



Giao



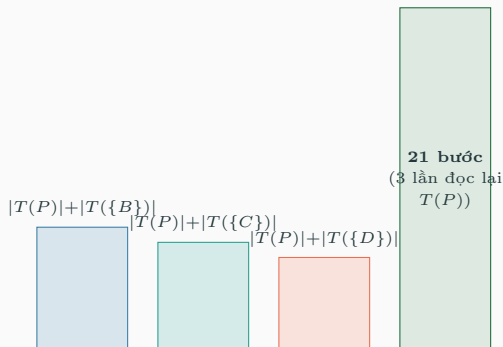
$i=5, j=4$: $9 = 9 \Rightarrow$ **xuất 9**, tăng cả hai. i vượt cuối $\Rightarrow$ dừng.

Mỗi phần tử của $\mathcal{T}(P_1)$ và $\mathcal{T}(P_2)$ chỉ bị con trỏ đi qua **đúng một lần** $\Rightarrow O(|\mathcal{T}(P_1)| + |\mathcal{T}(P_2)|)$ bước.

Vấn đề khi áp dụng down project cho Backtracking

Backtracking luôn thêm *đúng một item* mỗi lượt $\Rightarrow$ cặp tách duy nhất khả dụng là $P_1 = P$, $P_2 = \{e\}$. Với k candidate $e > \text{tail}(P)$:

$$O\left(\sum_{e > \text{tail}(P)} (|\mathcal{T}(P)| + |\mathcal{T}(\{e\})|)\right)$$



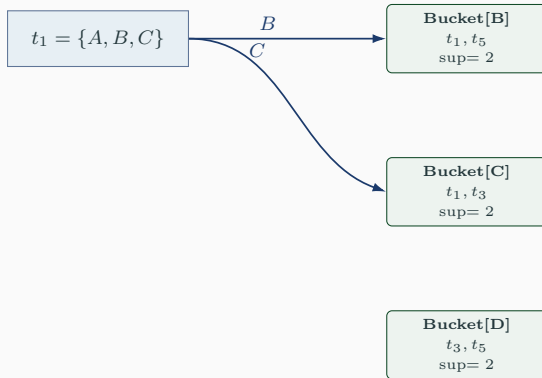
$|\mathcal{T}(P)|$ bị cộng lặp lại cho *từng* candidate, dù không đổi giữa các lần đó — đây là phần lãng phí mà OccurrenceDeliver loại bỏ.

OccurrenceDeliver: duyệt $\mathcal{T}(P)$ đúng một lần

```
OccurrenceDeliver(D, P):  
  for e > tail(P):  
    Bucket[e] := empty  
  for t in T(P):  
    for e in t, e > tail(P):  
      insert t into Bucket[e]  
  # Bucket[e] = T(P union {e})
```

$$O(|\mathcal{T}(P)| + \sum_{e > \text{tail}(P)} |\mathcal{T}(P \cup \{e\})|)$$

$$\mathcal{T}(\{A\}) = \{t_1, t_3, t_5\}$$



OccurrenceDeliver: duyệt $\mathcal{T}(P)$ đúng một lần

```
OccurrenceDeliver(D, P):
```

```
  for e > tail(P):
```

```
    Bucket[e] := empty
```

```
  for t in  $\mathcal{T}(P)$ :
```

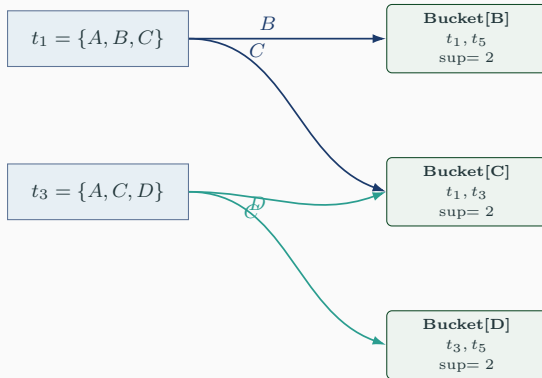
```
    for e in t, e > tail(P):
```

```
      insert t into Bucket[e]
```

```
  # Bucket[e] =  $\mathcal{T}(P \cup \{e\})$ 
```

$$O(|\mathcal{T}(P)| + \sum_{e > \text{tail}(P)} |\mathcal{T}(P \cup \{e\})|)$$

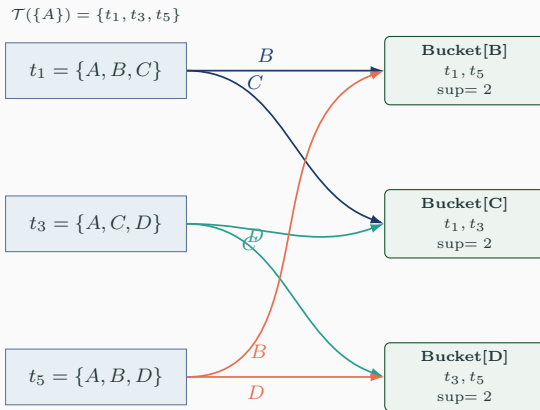
$\mathcal{T}(\{A\}) = \{t_1, t_3, t_5\}$



OccurrenceDeliver: duyệt $\mathcal{T}(P)$ đúng một lần

```
OccurrenceDeliver(D, P):  
  for e > tail(P):  
    Bucket[e] := empty  
  for t in  $\mathcal{T}(P)$ :  
    for e in t, e > tail(P):  
      insert t into Bucket[e]  
  # Bucket[e] =  $\mathcal{T}(P \cup \{e\})$ 
```

$$O(|\mathcal{T}(P)| + \sum_{e > \text{tail}(P)} |\mathcal{T}(P \cup \{e\})|)$$



OccurrenceDeliver: duyệt $\mathcal{T}(P)$ đúng một lần

```
OccurrenceDeliver(D, P):
```

```
  for e > tail(P):
```

```
    Bucket[e] := empty
```

```
  for t in  $\mathcal{T}(P)$ :
```

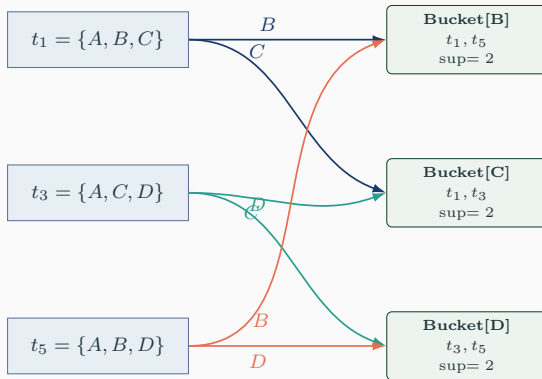
```
    for e in t, e > tail(P):
```

```
      insert t into Bucket[e]
```

```
  # Bucket[e] =  $\mathcal{T}(P \cup \{e\})$ 
```

$$O(|\mathcal{T}(P)| + \sum_{e > \text{tail}(P)} |\mathcal{T}(P \cup \{e\})|)$$

$\mathcal{T}(\{A\}) = \{t_1, t_3, t_5\}$



6 bước (so với 21 ở down project) — mỗi giao dịch chỉ "rải" đúng một lần, cho cả ba candidate cùng lúc.

Bất biến cần nhớ

$$\text{Bucket}[e] = \{t \in \mathcal{T}(P) : e \in t\} = \mathcal{T}(P \cup \{e\}) \quad \Rightarrow \quad \text{sup}(P \cup \{e\}) = |\text{Bucket}[e]|$$

Bucket là denotation của $P \cup \{e\}$ — dùng được ngay cho: kiểm tra minsup, đệ quy tiếp, và (slide sau) phát hiện item miễn phí trong HypercubeDecomposition — không cần tính lại gì cả.

Vì sao không dùng bitmap (Uno et al., 2004)

“This method has a disadvantage for sparse datasets ... not good for sparse large datasets.” LCMFreq gốc dùng mảng số nguyên. **Bản tối ưu của nhóm** đổi sang **BitArray**: tăng tốc mạnh trên dataset dày đặc (Chess, Pumsb), khiêm tốn hơn trên dataset thưa (Retail) — đúng như nhận định trên.

HypercubeDecomposition

Định nghĩa (Uno et al., 2004, Mục 3.3)

$H(P) = \{e > \text{tail}(P) : \mathcal{T}(P) = \mathcal{T}(P \cup \{e\})\}$. Kiểm tra $e \in H(P)$ chỉ cần so độ dài: $|\text{Bucket}[e]| = |\mathcal{T}(P)|$
— đã có sẵn sau OccurrenceDeliver.

$$P, S' = S \cup H(P)$$

Định nghĩa (Uno et al., 2004, Mục 3.3)

$H(P) = \{e > \text{tail}(P) : \mathcal{T}(P) = \mathcal{T}(P \cup \{e\})\}$. Kiểm tra $e \in H(P)$ chỉ cần so độ dài: $|\text{Bucket}[e]| = |\mathcal{T}(P)|$
— đã có sẵn sau OccurrenceDeliver.

$$P, S' = S \cup H(P)$$

Với mọi $Q \subseteq S'$: $P \cup Q$ frequent, $\text{sup} = \text{sup}(P)$
 $\Rightarrow$ xuất $2^{|S'|}$ itemset trong một lượt

Định nghĩa (Uno et al., 2004, Mục 3.3)

$H(P) = \{e > \text{tail}(P) : \mathcal{T}(P) = \mathcal{T}(P \cup \{e\})\}$. Kiểm tra $e \in H(P)$ chỉ cần so độ dài: $|\text{Bucket}[e]| = |\mathcal{T}(P)|$
— đã có sẵn sau OccurrenceDeliver.

$$P, S' = S \cup H(P)$$

Với mọi $Q \subseteq S'$: $P \cup Q$ frequent, $\text{sup} = \text{sup}(P)$
 $\Rightarrow$ xuất $2^{|S'|}$ itemset trong một lượt

“This saves about $2^{|H(P)|}$ times of the frequency counting.” — Uno et al. (2004). Đệ quy tiếp chỉ với $e \notin S'$; các item trong S' bị loại để tránh sinh lại.

- $e \in H(P) \Rightarrow \mathcal{T}(P \cup \{e\}) = \mathcal{T}(P)$; áp dụng liên tiếp cho mọi $e \in Q \subseteq S' \Rightarrow \mathcal{T}(P \cup Q) = \mathcal{T}(P)$.
- Đúng với mọi $Q \subseteq S'$, kể cả $Q = \emptyset$ (chính là P) và $Q = S'$.
- Không cần đệ quy riêng vào S' : đệ quy vào đó chỉ cho lại đúng những itemset đã xuất.

Thực giác

Hypercube không bỏ sót, không thêm sai: chỉ gom các node cùng denotation (cùng support) để xuất một lần, giảm số nhánh phải đi qua mà vẫn cho đúng tập frequent itemset như đệ quy đầy đủ.

Pseudo-code và ví dụ chạy tay

```
LCMFreqBase(P, tids_P, F, S):  
    support_P = length(tids_P)  
    Bucket[e] = [] for each e in F  
    for tid in tids_P:                # OccDeliver  
        for item in transaction[tid]:  
            if item in F:  
                push tid into Bucket[item]  
  
    H = {e in F : length(Bucket[e]) == support_P}  
    S_prime = S union H                # Hypercube  
    for Q subset of S_prime:  
        output P union Q with support_P  
  
    for e in F, e > tail(P), e not in S_prime:  
        if length(Bucket[e]) >= minsup:  
            LCMFreqBase(P+{e}, Bucket[e], F_next, S_prime)
```

Nhịp chạy tại node P :

1. Dựng bucket (OccDeliver)

```
LCMFreqBase(P, tids_P, F, S):
    support_P = length(tids_P)
    Bucket[e] = [] for each e in F
    for tid in tids_P:                # OccDeliver
        for item in transaction[tid]:
            if item in F:
                push tid into Bucket[item]

    H = {e in F : length(Bucket[e]) == support_P}
    S_prime = S union H                # Hypercube
    for Q subset of S_prime:
        output P union Q with support_P

    for e in F, e > tail(P), e not in S_prime:
        if length(Bucket[e]) >= minsup:
            LCMFreqBase(P+{e}, Bucket[e], F_next, S_prime)
```

Nhịp chạy tại node P :

1. Dựng bucket (OccDeliver)
2. $H(P) \rightarrow S'$

```
LCMFreqBase(P, tids_P, F, S):  
    support_P = length(tids_P)  
    Bucket[e] = [] for each e in F  
    for tid in tids_P:                # OccDeliver  
        for item in transaction[tid]:  
            if item in F:  
                push tid into Bucket[item]  
  
    H = {e in F : length(Bucket[e]) == support_P}  
    S_prime = S union H                # Hypercube  
    for Q subset of S_prime:  
        output P union Q with support_P  
  
    for e in F, e > tail(P), e not in S_prime:  
        if length(Bucket[e]) >= minsup:  
            LCMFreqBase(P+{e}, Bucket[e], F_next, S_prime)
```

Nhịp chạy tại node P :

1. Dựng bucket (OccDeliver)
2. $H(P) \rightarrow S'$
3. Xuất hàng loạt $P \cup Q$

```
LCMFreqBase(P, tids_P, F, S):  
    support_P = length(tids_P)  
    Bucket[e] = [] for each e in F  
    for tid in tids_P:                # OccDeliver  
        for item in transaction[tid]:  
            if item in F:  
                push tid into Bucket[item]  
  
    H = {e in F : length(Bucket[e]) == support_P}  
    S_prime = S union H                # Hypercube  
    for Q subset of S_prime:  
        output P union Q with support_P  
  
    for e in F, e > tail(P), e not in S_prime:  
        if length(Bucket[e]) >= minsup:  
            LCMFreqBase(P+{e}, Bucket[e], F_next, S_prime)
```

Nhịp chạy tại node P :

1. Dựng bucket (OccDeliver)
2. $H(P) \rightarrow S'$
3. Xuất hàng loạt $P \cup Q$
4. Đặt quy ngoài S'

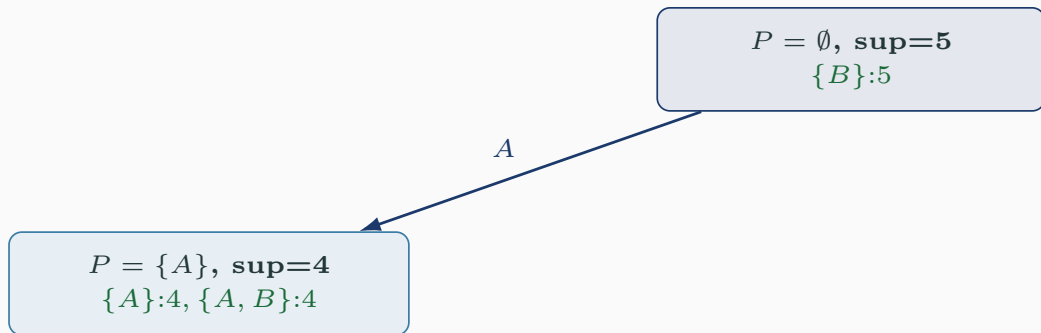
TID	Items
T_1	A, B, C, D
T_2	A, B, D
T_3	A, B, C
T_4	B, C
T_5	A, B, C, D

$\sigma = 2$, $P = \emptyset$, $\mathcal{T}(\emptyset) =$ cả 5 giao dịch.

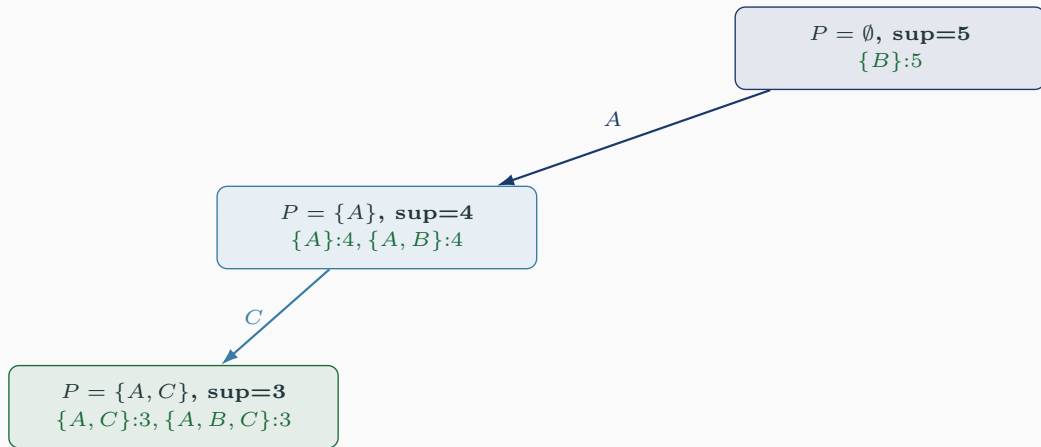
e	Bucket[e]	sup
A	T_1, T_2, T_3, T_5	4
B	T_1, T_2, T_3, T_4, T_5	5
C	T_1, T_3, T_4, T_5	4
D	T_1, T_2, T_5	3

$|\text{Bucket}[B]| = 5 = |\mathcal{T}(\emptyset)| \Rightarrow B \in H(\emptyset)$. Xuất ngay $\{B\}$:5, đệ quy tiếp chỉ với A, C, D .

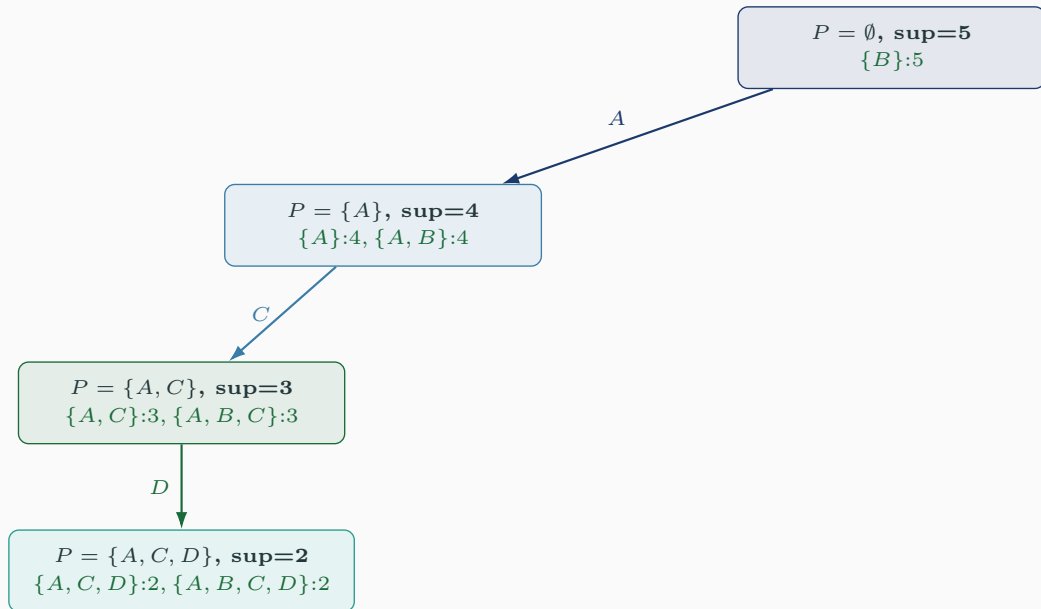
$$P = \emptyset, \text{ sup}=5$$
$$\{B\}:5$$



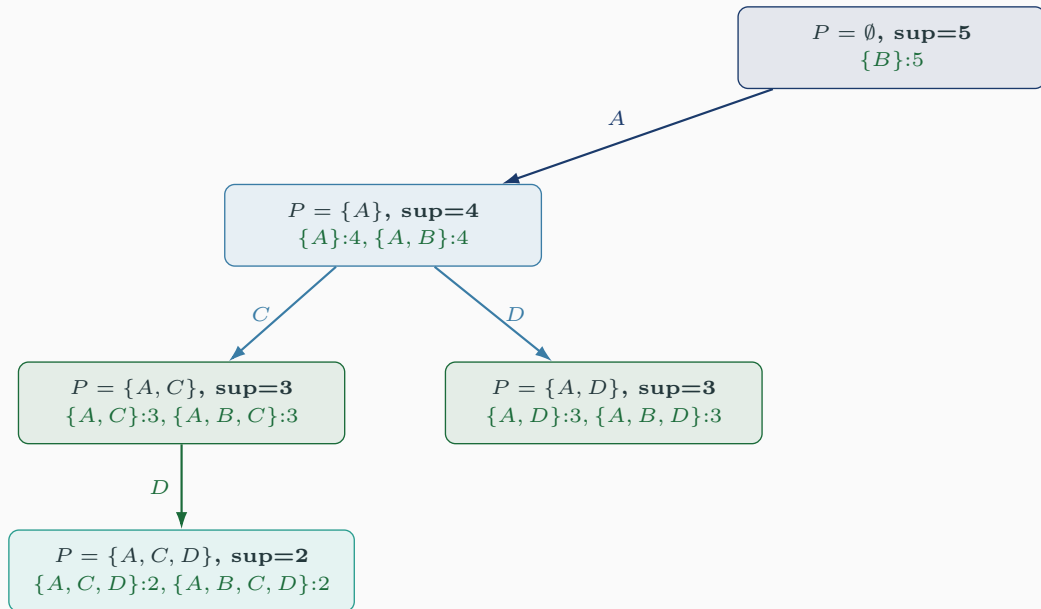
Cây đệ quy trên ví dụ (theo đúng thứ tự DFS)



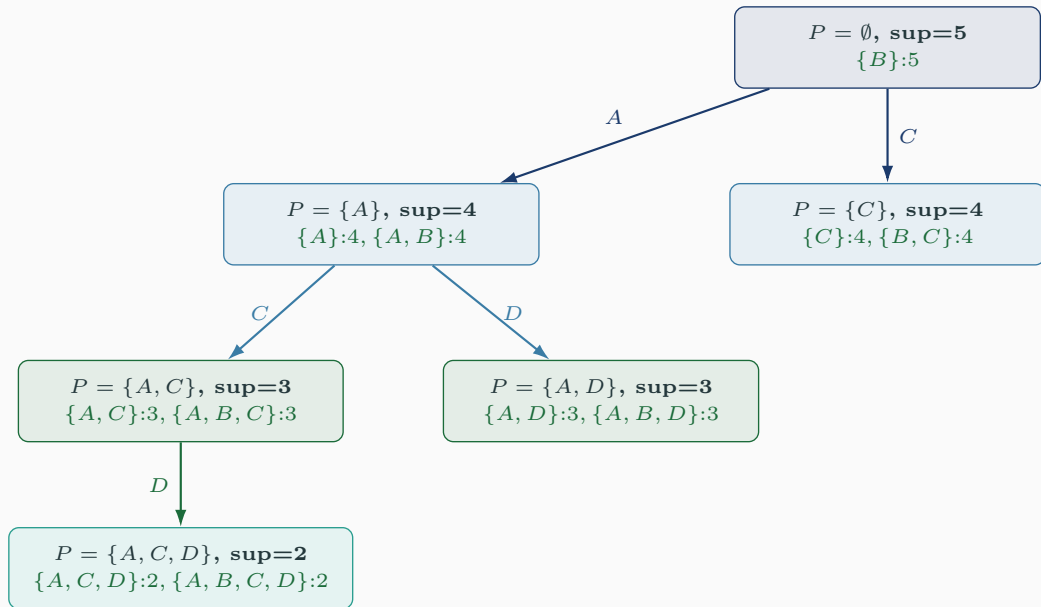
Cây đệ quy trên ví dụ (theo đúng thứ tự DFS)



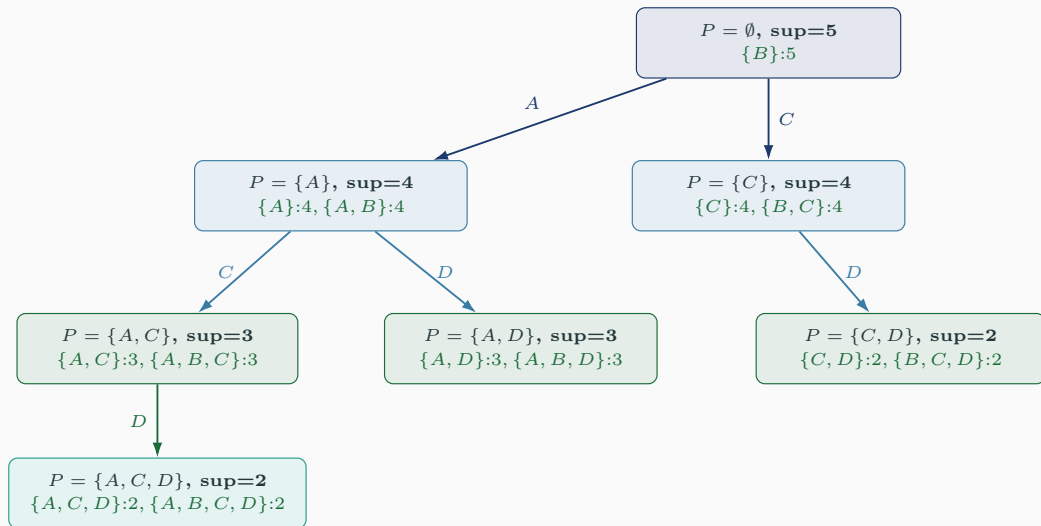
Cây đệ quy trên ví dụ (theo đúng thứ tự DFS)



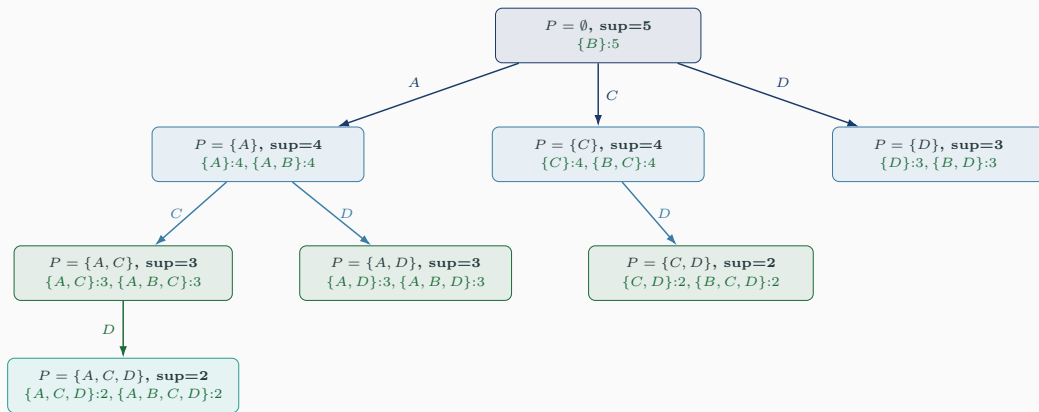
Cây đệ quy trên ví dụ (theo đúng thứ tự DFS)



Cây đệ quy trên ví dụ (theo đúng thứ tự DFS)



Cây đệ quy trên ví dụ (theo đúng thứ tự DFS)



Chỉ 7 node thật sự ($\emptyset, A, C, D, AC, AD, CD, ACD$); 8 itemset chứa B được xuất kèm bằng Hypercube, không tồn nhánh đệ quy riêng.

Cài đặt Julia

```
support_P = length(tids_P)
local_buckets = Dict{Int, Vector{Int}}{
    item => Int[] for item in freq_items)

for tid in tids_P
    for item in transactions[tid]
        haskey(local_buckets, item) &&
            push!(local_buckets[item], tid)
    end
end
```

OccurrenceDeliver: với mỗi giao dịch trong $\mathcal{T}(P)$,
đẩy TID vào bucket của mọi candidate nó chứa.

```
H_P = filter(e ->
    length(local_buckets[e]) == support_P,
    freq_items)
S_prime = sort!(collect(union(S, H_P)))
```

HypercubeDecomposition: giữ lại e có bucket dài
bằng support_P (không mất giao dịch nào khi
thêm e).

```
for Q in _all_subsets_base(S_prime)
    push!(result, FreqItemset(
        sort!([p[1:plen]; Q]), support_P))
end
```

Xuất mọi $P \cup Q$, $Q \subseteq S'$, một lần cho cả $2^{|S'|}$ tổ hợp.

```
for e in freq_items
    e <= tail_P && continue
    e in S_prime_set && continue
    tids_Pe = local_buckets[e]
    length(tids_Pe) < minsup && continue
    _hypercube_base!(p, plen+1, tids_Pe, ...)
end
```

Đệ quy chỉ vào $e \notin S'$, $e > \text{tail}(P)$, đủ minsup.

Khung thuật toán giữ nguyên ở cả hai bản; khác biệt duy nhất là cách biểu diễn denotation và phép giao.

Baseline — bám sát bài báo

`Vector{Int}` cho denotation; `push!` TID vào bucket. Đơn giản, nhưng cấp phát theo chỉ số item lớn nhất $\Rightarrow$ OOM trên dataset nhiều item (Retail: 16 470 item).

Optimized — BitArray

Denotation là `BitArray`; giao = AND theo bit, đếm support = count (POPCNT phần cứng).

```
tidset_Pe = tidset_P .& tidsets_full[e]
```

Theo đúng nhận định của Uno et al. (2004): bitmap lợi nhất trên dataset *dày đặc* (Chess, Pumsb), khiêm tốn trên dataset *thưa* (Retail) — xem số liệu ở phần sau.

Đánh giá thực nghiệm

9 dataset chuẩn từ FIMI, đối chiếu SPMF (Java) trên cùng máy, lấy thời gian trung vị qua nhiều lần lặp.

Dataset	#Trans.	#Item	Độ dài TB	Đặc điểm
Chess	3 196	75	37.0	dày đặc, itemset dài
Connect	67 557	129	43.0	dày đặc, rất nén
Mushroom	8 124	119	23.0	dày đặc, nhiều item
Pumsb	49 046	2 113	74.0	dày đặc, độ dài rất lớn
Accidents	340 183	468	33.8	rất lớn, trung gian
Retail	88 162	16 470	10.3	thưa, dữ liệu thực
T10I4D100K	100 000	870	10.1	tổng hợp, thưa
T40I10D100K	100 000	942	39.6	tổng hợp, thưa, dài

Chi phí node P : $O\left(\sum_{T \in \mathcal{T}(P)} |T \cap F|\right)$ (không duyệt lại như down project)

LCMFreq là **output-sensitive**: thời gian gắn với số kết quả phải xuất.

Yếu tố tăng	Vì sao ảnh hưởng
Số giao dịch $ \mathcal{D} $	mỗi node duyệt/AND nhiều bit hơn
Số item $ \mathcal{I} $	nhiều candidate $\Rightarrow$ nhiều nhánh đệ quy
Minsup thấp	cây ít bị cắt tĩa, số kết quả tăng theo hàm mũ
Dataset dày đặc	nhiều item cùng denotation $\Rightarrow$ Hypercube + BitArray phát huy tối đa

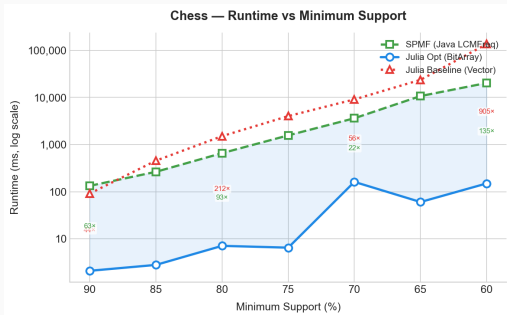
Đối chiếu số lượng frequent itemset với SPMF

Dataset	minsup	Julia	SPMF
Chess	75%	20 993	20 993
Mushroom	30%	2 735	2 735
Pumsb	90%	2 607	2 607
Retail	2%	55	55
T10I4D100K	0.8%	494	494

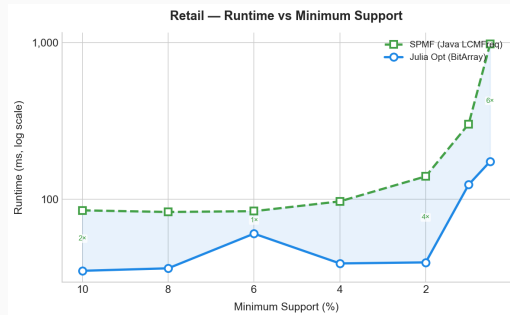
Khớp 100%

Khớp tuyệt đối trên toàn bộ 9 dataset — không thiếu, không thừa, không sai support của bất kỳ itemset nào. Unit test còn so support từng itemset trên toy dataset nhỏ (ví dụ chạy tay ở trên).

Thời gian chạy theo ngưỡng minsup



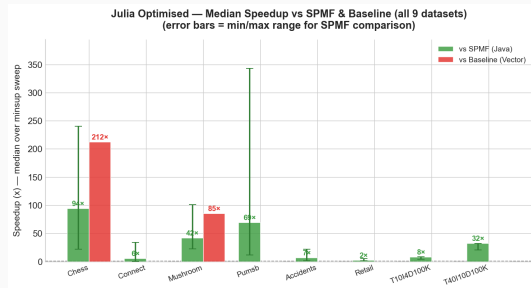
Chess (dày đặc)



Retail (thưa)

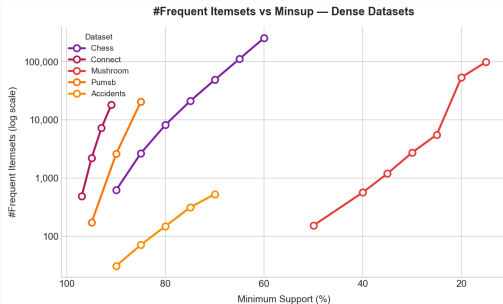
Chess: nhanh hơn SPMF 76–141×. Retail: 2–6× (denotation đã nhỏ sẵn). Baseline OOM trên Retail (16 470 item).

Dataset	Trung vị	Min	Max
Chess	135×	22×	241×
Pumsb	343×	12×	343×
Mushroom	58×	23×	101×
T40I10D100K	32×	21×	32×
Connect	6×	1×	34×
Retail	2×	1×	6×



Pumsb cao nhất nhờ giao dịch dài (74 item) $\Rightarrow$ AND bit khuếch đại lợi thế.

Số lượng frequent itemset & bộ nhớ

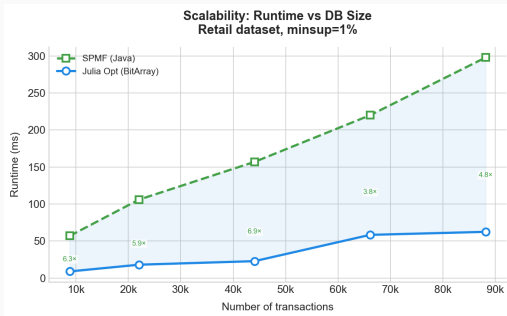


Số itemset theo minsup (dataset dày đặc)

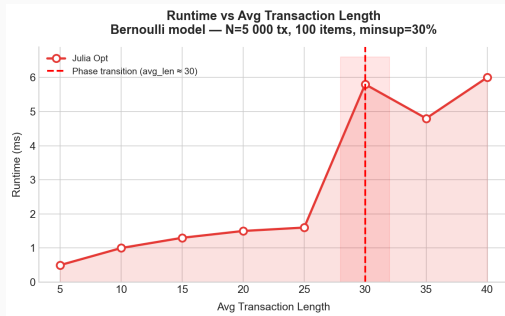
Dataset	Opt (MiB)	Baseline	Giảm
Chess	31.87	857.23	26.9×
Mushroom	10.98	161.57	14.7×
Retail	20.68	OOM	N/A
T10I4D100K	1 215.2	OOM	N/A

BitArray: 1 bit/transaction (vs. 8 byte/TID ở `Vector{Int}`) $\Rightarrow$ tiết kiệm $\sim 64\times$ mỗi denotation.

Khả năng mở rộng & ảnh hưởng độ dài giao dịch



Retail, lấy mẫu theo tỉ lệ — thời gian chạy gần tuyến tính theo số giao dịch.



Dataset tổng hợp: khi độ dài TB vượt ngưỡng pha chuyển (≈ 30), itemset phổ biến xuất hiện đột ngột.

Ứng dụng: phân tích giỏ hàng

Với itemset X frequent, $Y \subset X$:

$$\text{conf}(Y \Rightarrow X \setminus Y) = \frac{\text{sup}(X)}{\text{sup}(Y)} \geq \text{minconf}, \quad \text{lift} > 1 \Rightarrow \text{đồng xuất hiện nhiều hơn ngẫu nhiên.}$$

Retail (88 162 giao dịch): minsup= 0.5% ($\sigma = 441$) $\rightarrow$ 580 frequent itemset; minconf= 60% $\rightarrow$ 312 association rule.

Vế trái	Vế phải	Support	Conf.	Lift
$\{38, 39\}$	$\{41\}$	1.01%	0.89	8.7
$\{38\}$	$\{39, 41\}$	1.01%	0.82	8.1
$\{39\}$	$\{38\}$	1.18%	0.73	7.2
$\{32\}$	$\{48\}$	0.72%	0.78	6.4

Nhóm $\{38, 39, 41\}$: lift > 7 — ứng viên mạnh cho bán theo gói / bố trí gần nhau.

Kết luận

Đã hoàn thành

- Backtracking + OccurrenceDeliver + Hypercube theo Uno et al. (2004)
- **Bản tối ưu BitArray**, unit test tự động
- Khớp 100% với SPMF (9 dataset)
- Ứng dụng giỏ hàng trên Retail

Giới hạn

- Bộ nhớ còn cao trên T10I4D100K (~1.2 GiB)
- Chưa dùng diffset
- Chưa song song hóa DFS

Hướng phát triển

- Diffset để giảm bộ nhớ
- Đa luồng cho nhánh top-level
- Mở rộng sang closed/maximal itemset

Tóm lại

Backtracking tiết kiệm bộ nhớ so với Apriori; OccurrenceDeliver khắc phục điểm yếu đếm support của backtracking; Hypercube giảm số lần đếm lặp lại. Bản tối ưu BitArray giữ nguyên khung sinh itemset, chỉ đổi cấu trúc denotation — **đóng góp riêng của nhóm**, không sao chép từ SPMF (SPMF chỉ dùng để đối chiếu kết quả).

1. T. Uno, M. Kiyomi, H. Arimura. *LCM ver.2: Efficient Mining Algorithms for Frequent/Closed/Maximal Itemsets*. IEEE ICDM Workshop on FIMI, Brighton, UK, 2004.
2. R. Agrawal, R. Srikant. *Fast Algorithms for Mining Association Rules*. Proc. 20th VLDB, pp. 487–499, 1994.
3. J. Han, J. Pei, Y. Yin. *Mining Frequent Patterns without Candidate Generation*. Proc. ACM SIGMOD, pp. 1–12, 2000.
4. M. J. Zaki et al. *New Algorithms for Fast Discovery of Association Rules*. Proc. 3rd KDD, pp. 283–286, 1997.

Cảm ơn thầy/cô và các bạn!

Câu hỏi?

Nhóm 01, CSC14004 | Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM